UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	M.Ikbal Pratama 2428240135
Kelas	SI5C
Judul project / topik API	Informasi Beasiswa Topik 36
Tautan repositori	mikbalpratama/si5c_kuis1_mikbalpratama
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	18-10-2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

Jelaskan dalam dua sampai tiga kalimat: topik API Anda, entitas utamanya, dan teknologi yang dipakai (misalnya Express.js dengan struktur Route-Controller-Model).

API yang saya buat bertopik Informasi Beasiswa, yang digunakan untuk mengelola data service order, seperti nama pelanggan, jenis service, biaya, jenjang nama pelanggaan, dan status. Entitas utamanya adalah service order, dengan fitur untuk menampilkan, menambahkan, mengubah, menghapus, dan memfilter data berdasarkan jenjang orders. Teknologi yang digunakan adalah Node.js dan Express.js dengan struktur sederhana yang ditulis langsung dalam file app.js, serta diuji menggunakan Postman dan Thunder Client.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	{{base_url}}/service-orders		200	Mencari semua service order
2	GET	{{base_url}}/service-orders/{{serviceId}}		200	Mencari semua servis order berdasarkan id
3	POST	{{base_url}}/service-orders	12345678	201	Membuat servis order baru
4	PUT	{{base_url}}/service-orders/{{serviceId}}	12345678		Mengedit servis order
5	DELETE	{{base_url}}/service-orders/{{serviceId}}	12345678		Menghapus servis order
6					
7					
8					

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

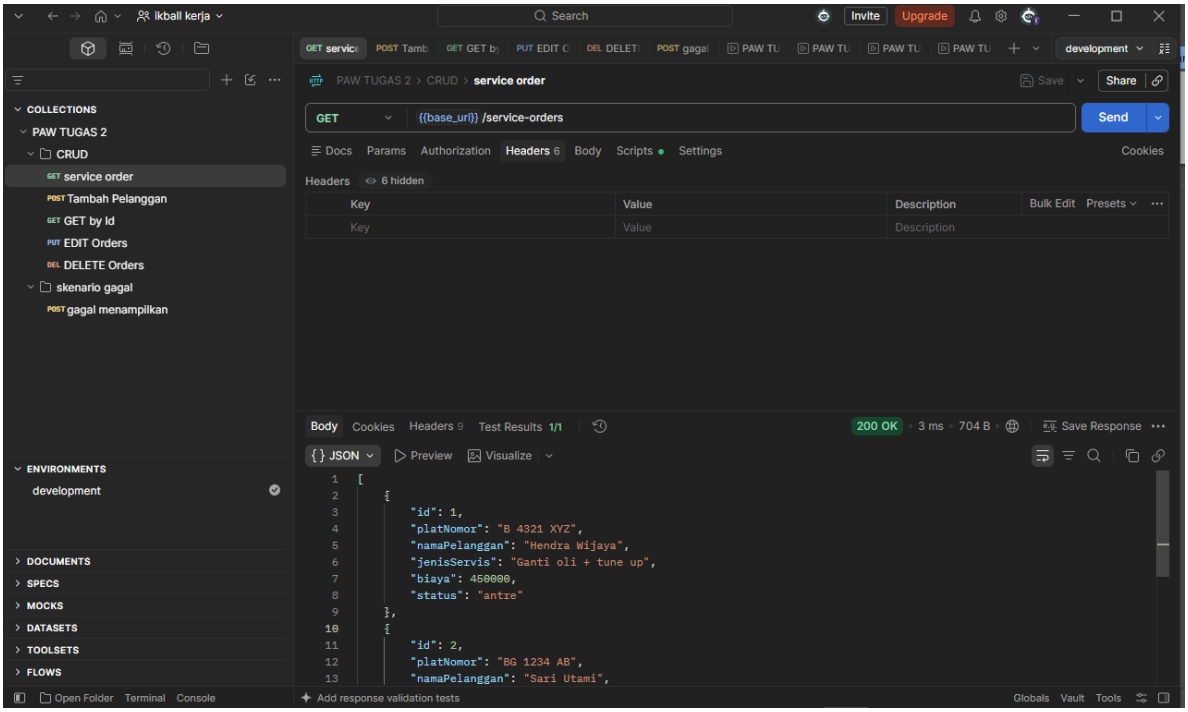
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	{{base_url}}/service-orders	Alamat server
2	api_key	12345678	Nilai header kunci API
3	serviceId	(kosong, diisi script)	ID dari respons POST
4			

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Jelaskan cara Anda mengelompokkan request (misalnya folder CRUD dan folder skenario gagal) dan mengapa urutannya seperti itu.

Collection dibuat dengan nama PAW II lalu tambahkan folder CRUD, lalu klik PAW II dan tambahkan folder Skenario Gagal selanjutnya tekan CRUD tambahkan beberapa request seperti diatas dan untuk Skenario gagal juga sama.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Get by id	GET	{{base_url}}/service-orders/{{serviceId}}	CRUD
2	Tambah Pelanggan	POST	{{base_url}}/service-orders	CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
3	Edit orders	PUT	{{base_url}}/service-orders/{{serviceId}}	CRUD
4	Delete orders	DELETE	{{base_url}}/service-orders/{{serviceId}}	CRUD
5	POST gagal menampilkan	post	{{base_url}}/service-orders	Skenario gagal
6				
7				
8				
9				
10				

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

1	GET by Id
	{{base_url}}/service-orders/{{serviceId}}
		200	pm.test('Status code adalah 200', *function* () {
			  pm.response.to.have.status(200);
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

2	POST Tambah Pelanggan {{base_url}}/service-orders	201	{
			
			"platNomor": "B 4321 PPK",
			"namaPelanggan": "JOKOWI Wijaya",
			"jenisServis": "Ganti MESIN + tune up",
			"biaya": 450000,
			"status": "antre"
			}
			
			pm.test("Status code adalah 201", *function* () {
			pm.response.to.have.status(201);
			});
			
			pm.test("Response memiliki ID service", *function* () {
			const jsonData = pm.response.json();
			
			pm.expect(jsonData.data).to.have.property("id");
			});
			const jsonData = pm.response.json();
			pm.environment.set("serviceId", jsonData.data.id);

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

3	PUT Edit Orders
	{{base_url}}/service-orders/{{serviceId}}
		200 OK	{
			
			"platNomor": "BG 1266 KP",
			"namaPelanggan": "IKBAL PRATAMA",
			"jenisServis": "Ganti MASTER rem",
			"biaya": 300000,
			"status": "selesai"
			}
			
			pm.test("Status code adalah 200", *function* () {
			pm.response.to.have.status(200);
			});

4	Delete Orders
	{{base_url}}/service-orders/{{serviceId}}
		200 OK	pm.test('Status code adalah 204', *function* () {
			pm.response.to.have.status(204);
			});
			
			pm.test('Response tidak memiliki isi', *function* () {
			pm.expect(pm.response.text()).to.eql('');
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

5	gagal menampilkan	400	{pm.test('Status code adalah 400', *function* () {
			pm.response.to.have.status(400);
			});
			
			pm.test('Pesan error menyebut field wajib', *function* () {
			pm.expect(pm.response.json().message).to.include('wajib diisi');
			});

6			
7			
8			
9			
10			

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: {{base_url}}/service-orders/{{serviceId}}
pm.test('Status code adalah 200', function () {
  pm.response.to.have.status(200);
});
```

Penjelasan:

Jelaskan fungsi setiap `pm.test`: apa yang diperiksa dan mengapa pemeriksaan itu penting.

Pengujian pertama digunakan untuk memastikan bahwa request GET berhasil dijalankan dan menghasilkan status code 200. Pengujian kedua digunakan untuk memeriksa apakah ID service order pada respons sesuai dengan ID yang tersimpan di Environment Postman. Dengan pengujian ini, kita dapat memastikan bahwa API mengembalikan data service order yang sesuai dengan ID yang diminta.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: {{base_url}}/service-orders

pm.test("Status code adalah 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Response memiliki ID service", function () {
    const jsonData = pm.response.json();

    pm.expect(jsonData.data).to.have.property("id");
});
const jsonData = pm.response.json();
pm.environment.set("serviceId", jsonData.data.id);
```

Penjelasan:

Jelaskan bagaimana data dari respons disimpan, request mana yang memakainya, dan apa yang terjadi bila urutan request ditukar.

Pengujian pertama memastikan bahwa proses penambahan beasiswa berhasil dengan status code 201. Pengujian kedua memeriksa apakah respons API memiliki ID service order yang akan digunakan pada request berikutnya. Setelah itu, ID disimpan ke variabel serviceId pada Environment Postman menggunakan pm.environment.set().

Variabel tersebut digunakan oleh request GET service-order by ID, PUT Ubah Orders, dan DELETE Orders. Jika urutan request ditukar dan POST belum dijalankan, variabel ID mungkin masih kosong atau berisi ID dari pengujian sebelumnya, sehingga request berikutnya dapat gagal atau mengakses data yang tidak sesuai.

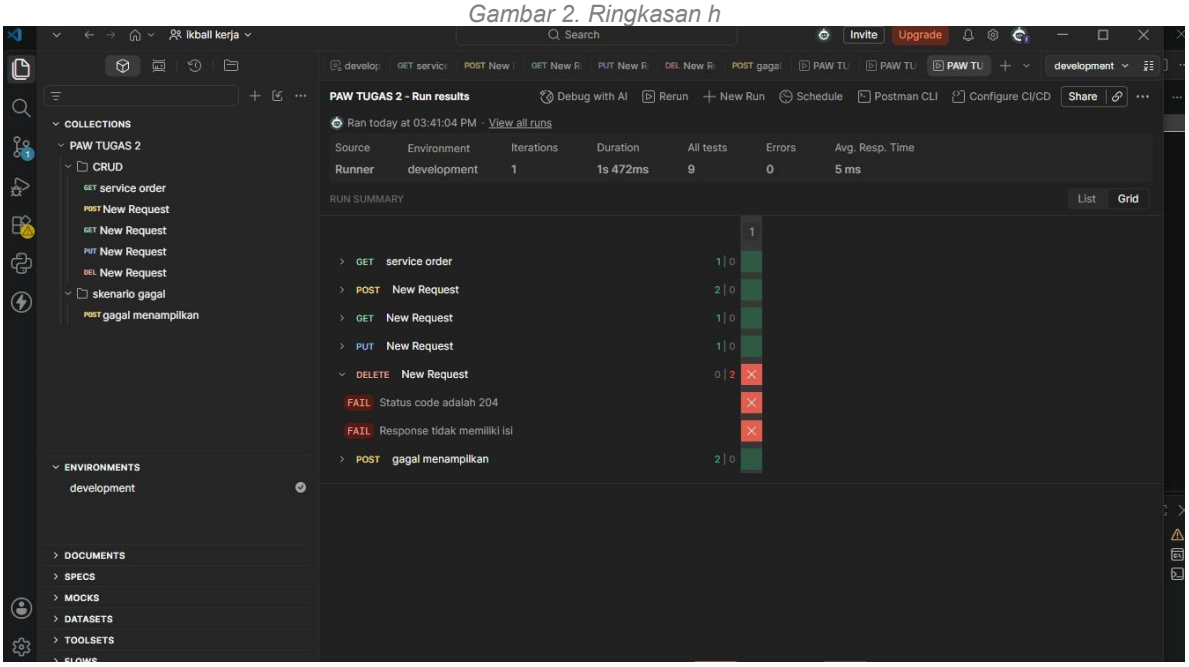
Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.

Gambar 2. Ringkasan h



asil Collection Runner

Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Jelaskan hasilnya: berapa request dan test yang dijalankan, apakah semuanya lulus, dan apa yang Anda lakukan sebelum menjalankan Runner (misalnya menyalakan ulang server).

Ada 6 request dan hasilnya ada yg error

4.2 Rekap angka hasil

Nama Collection	PAW II
Environment yang dipakai	Development
Jumlah request dijalankan	6
Jumlah test lulus	5
Jumlah test gagal	1
Tanggal dan jam pengujian	18-10-2026 jam 3 sore

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1				
2				

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot di sini]

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	mikbalpratama/si5c_kuis1_mikbalpratama
Lokasi berkas Collection (.json)	
Lokasi berkas Environment (.json)	
Berkas Environment tidak memuat kunci rahasia sungguhan	[ ] Ya [ ] Tidak
Repositori dapat dibuka dosen	[ ] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Tulis jawaban Anda di sini.

Assertion yang paling berguna menurut saya adalah pengecekan status code dan struktur data JSON. Dengan assertion tersebut, saya bisa mengetahui apakah API memberikan respons yang sesuai dan apakah data yang dikembalikan sudah benar, bukan hanya sekadar request berhasil dikirim.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Tulis jawaban Anda di sini.

Saat menguji API secara manual, saya harus mengirim request satu per satu, sedangkan dengan Collection Runner semua request dapat dijalankan sekaligus sehingga lebih praktis untuk memeriksa hasil pengujian. Bagian yang paling sulit bagi saya adalah menulis test script dan memastikan variabel ID dapat digunakan pada request berikutnya. Saya mengatasinya dengan mencoba script secara bertahap dan memeriksa hasilnya di Postman.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	
2	Setiap endpoint API saya ada di Collection	
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	
4	Setiap request punya assertion status code dan struktur JSON	
5	Nama setiap test berupa kalimat yang menyatakan harapan	
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	
7	Server dinyalakan ulang sebelum Runner dijalankan	
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	
9	Berkas Environment tidak memuat kunci rahasia sungguhan	
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen